

LZM Algorithm

OLADAYO OYEDEJI (APRIL 24, 2026)

Contents

1	Introduction and Background	2
1.1	Context Probability Model	2
1.2	Range Encoding	3
2	Math and Algorithm Concept	3
2.1	Optimal Entropy	3
2.2	Conditional Probability (Markov Chain)	4
2.3	Recursive Interval Subdivision	5
2.4	Adaptive Update	5
2.5	Proof	6
3	Implementation	6
3.1	LzmaEncode	6
3.2	Encode Literal	7
3.3	Range Encode Bit	8

1 Introduction and Background

LZMA (Lempel-Ziv-Markov chain Algorithm) was developed by Igor Pavlov around 1998 as part of the 7-Zip compression system. It improves on earlier LZ based methods like LZ77(developed by Abraham Lempel and Jacop Ziv), which compress data by replacing repeated byte sequences with references to earlier occurrences.

LZMA still utilizes Lempel-Ziv idea of dictionary-based compression, but it adds a much powerful statistical model. Instead of only finding repeated substring, it predicts the probability of the next byte based on context, meaning the sequence of previous bytes, This is the "Markov chain" part of the name: the assumption that the next symbol depends on recent history.

Unlike LZ77 alone, LZMA combines this context modeling with range encoding (a form of entropy coding similar to arithmetic coding), where more probable symbols are encoded with fewer bits and less probable symbols use more bits.

This step is based on LZ77-style matching, where the input stream is represented using two types of symbols: literals and matches.

Literals are single bytes that have no sufficiently long match in the previous data window. They are stored directly when no useful repetition is found.

Match sequences represent repeated byte patterns found earlier in the data. A match is encoded using:

- The *length* of the repeated sequence
- The *distance* (how far back in the data the match occurs)

Matches are typically only used when the repeated sequence is longer than a small threshold (often 2 or more bytes), since shorter matches are not efficient to encode.

1.1 Context Probability Model

This step uses a Markov-style context model to estimate the probability of the next symbol based on previous symbols.

Instead of treating all bytes equally, the model conditions predictions on the recent history (context). For example, after seeing a phrase like:

"The capital of France is "

the model assigns a higher probability to words such as "Paris" because similar contexts have appeared in training data or prior observations.

The context used depends on whether the current symbol is a literal or part of a match. Over time, the model adapts its probability estimates based on observed frequencies, improving prediction accuracy during compression.

1.2 Range Encoding

The final step converts predicted probabilities into a compact bitstream using range encoding (a form of entropy coding similar to arithmetic coding).

Symbols with higher predicted probability are encoded using fewer bits, while less likely symbols require more bits. This allows the compressor to achieve high efficiency by aligning bit usage with statistical likelihoods.

2 Math and Algorithm Concept

LZMA and many modern compression algorithms are influenced by the concept of Shannon entropy, which represents the theoretical lower bound on the average number of bits required to encode data based on symbol probabilities.

2.1 Optimal Entropy

The entropy of a discrete random variable X is defined as:

$$H(X) = - \sum_{i=1}^n P(x_i) \lg_2 P(x_i)$$

This value represents the minimum expected number of bits required per symbol for optimal encoding.

In a simple compression system where each bit has equal probability,

$$P(x_i) = 0.5$$

then:

$$H(X) = 1 \text{ bit per symbol}$$

meaning no compression is possible because the data is already maximally unpredictable.

However, when symbols occur with unequal probabilities, entropy becomes less than 1 bit per symbol on average, allowing compression algorithms such as LZMA to encode more frequent symbols using fewer bits through entropy coding techniques like range encoding.

2.2 Conditional Probability (Markov Chain)

LZMA makes use of conditional probability to predict future symbols based on previously encoded data. This can be written as

$$P(X | S)$$

where X represents the next symbol and S represents the current encoder state determined by recent encoding history.

One component of this context modeling uses the high-order bits of the previously encoded literal byte. For example, when three bits of the previous literal are used, this produces

$$2^3 = 8$$

possible literal contexts. These contexts form an 8-state model that improves prediction accuracy when encoding literal bytes.

In addition to literal contexts, LZMA maintains a 12-state finite-state model that tracks whether recent outputs were literals, matches, or repeated matches. These states influence the probability of whether the next encoded symbol will be a literal or a match.

For example:

state 0–6: recent history dominated by literals

state 7–11: recent history dominated by matches

By conditioning symbol probabilities on recent encoding history, LZMA adapts dynamically to the structure of the input data and improves compression effi-

ciency.

2.3 Recursive Interval Subdivision

During range encoding, the current interval is subdivided according to the predicted probability of the next bit. Let

$$\text{bound} = \left\lfloor \frac{R_{old}}{2048} \right\rfloor \times P$$

Then the interval update rule becomes

$$(L_{new}, R_{new}) = \begin{cases} (L_{old}, \text{bound}) & \text{if bit} = 0 \\ (L_{old} + \text{bound}, R_{old} - \text{bound}) & \text{if bit} = 1 \end{cases}$$

where L represents the lower bound of the interval and R represents the size of the current interval. This recursive subdivision progressively narrows the interval so that it uniquely represents the encoded bit sequence.

2.4 Adaptive Update

The probability model in LZMA is updated adaptively using an exponentially weighted moving average (EWMA). The update rule is

$$P_{t+1} = P_t + \alpha(B_t - P_t)$$

where P_t represents the current probability estimate, B_t is the observed bit value, and α is the learning rate. This update gradually shifts the probability estimate toward recently observed values while retaining information from previous observations.

In practice, LZMA implements this update using fixed-point integer arithmetic, allowing efficient real-time probability adaptation during range encoding. This adaptive mechanism enables the encoder to respond quickly to changing symbol statistics and improves compression efficiency.

2.5 Proof

3 Implementation

My implementation is written in C++.

In LZMA, probabilities are represented using 11-bit unsigned integers in the range

$$0 < \text{prob} < 2048$$

where the value `prob` represents the estimated probability that the next bit is 0:

$$P(\text{bit} = 0) \approx \frac{\text{prob}}{2048}$$

and therefore

$$P(\text{bit} = 1) = 1 - P(\text{bit} = 0).$$

Values close to 2048 indicate a high likelihood of a 0 bit, while values close to 0 indicate a high likelihood of a 1 bit. This fixed-point probability representation allows efficient implementation of adaptive probability updates using integer arithmetic during range encoding.

3.1 LzmaEncode

```
Encode()
    pos = 0

    while (pos < size)
        dist, len = get_match(pos)

        if (len < 2)
            litNext[12] = {0,0,0,0,1,2,3,4,5,6,4,5}

            range_encode_bit(ismatch[state][posState], 0)
            encode_literal(pos)

            state = litNext[state]
```

```

        pos++

    else
        range_encode_bit(ismatch[state][posState], 1)

        encode_length(posState, len)
        encode_distance(dist - 1, len)

        state = (state < 7) ? 7 : 10
        pos += len

range_flush()

```

The encoder determines whether the next symbol is encoded as a literal or a match using the probability model `ismatch[state][posState]`. The state variable tracks recent encoding history and influences future probability predictions.

3.2 Encode Literal

```

encode_literal(byte)
    litState = prev >> (8 - lc)
    probs = litProbs + 768 * litState

    node = 1

    for i = 7 downto 0
        bit = (byte >> i) & 1
        range_encode_bit(probs[node], bit)
        node = (node << 1) | bit

```

The literal context is determined using the upper bits of the previously encoded byte. This context selects one section of the literal probability array.

Each literal context contains 768 probability values organized as three binary trees of depth 8. A binary tree of depth 8 requires

$$2^0 + 2^1 + \dots + 2^7 = 255$$

probability nodes to encode all possible decision paths for an 8-bit symbol.

LZMA uses three such trees per context:

- one tree for normal literal encoding

- two additional trees for matched literal encoding

Matched literal encoding is used when the encoder predicts the literal based on a previously matched byte from the dictionary. The encoder then conditions probability selection on whether the predicted match bit agrees with the literal bit. This improves compression performance by incorporating additional contextual information during encoding.

3.3 Range Encode Bit

```
range_encode_bit(prob, bit)
    bound = (range >> 11) * prob

    if bit
        low += bound
        range -= bound
        prob -= prob >> 5
    else
        range = bound
        prob += (2048 - prob) >> 5

    if range < (1 << 24)
        range <<= 8
        shiftlow()
```

Each encoded bit subdivides the current interval according to the estimated probability value. The interval is then renormalized whenever its size falls below 2^{24} to maintain sufficient precision during encoding. The probability value is updated adaptively using a fixed-point exponential moving average after each encoded bit.